

Ampalis Team

Gruppo 2 · Ingegneria del Software · a.a. 2026/2027

Norme di Progetto

Regole di lavoro adottate da Ampalis Team: strumenti, convenzioni e procedure condivise.

Versione	1.3.0
Data	2026-11-11
Stato	Redatto
Uso	Interno
Destinatari	Ampalis Team, Prof. Tullio Vardanega, Prof. Riccardo Cardin
Redazione	Pietro Vigolo, Matteo Cetraro
Verifica	Matteo Cetraro, Pietro Vigolo
Approvazione	Ivan Valente

ampalisteam@gmail.com

Registro delle modifiche

Versione	Data	Autore	Verificatore	Descrizione
1.3.0	2026-11-11	Matteo Cetraro	Pietro Vigolo	Riscritte le regole su pull request e verifica secondo il verbale interno del 2026-11-11: diario di bordo del Responsabile senza verifica, approvazione per il rilascio nella stessa pull request della verifica, un solo verificatore per prodotto; vista web generata dai PDF, senza elenco dei documenti
1.2.0	2026-11-04	Pietro Vigolo	Matteo Cetraro	Riscritte le regole di versionamento, verifica e approvazione per il rilascio secondo l'esito dell'aggiudicazione del 2026-11-03; aggiornato il logo dell'Università e riordinato il registro: la versione 1.0.0 registra solo l'approvazione per il rilascio
1.1.0	2026-10-30	Pietro Vigolo	Matteo Cetraro	Allineate le regole su consegne e incontri con il proponente ai verbali interni del 2026-10-28 e del 2026-10-29; corretti riferimenti, descrizione della cartella `template` ed esempio di messaggio di commit
1.0.0	2026-10-29	Ivan Valente	–	Approvazione per il rilascio
0.2.0	2026-10-29	Pietro Vigolo	Matteo Cetraro	Completato per la candidatura: aggiunte le regole sulla vista web e aggiornata la struttura del repository della documentazione
0.1.0	2026-10-28	Pietro Vigolo	Matteo Cetraro	Prima stesura del documento

Indice

1 Introduzione **3**

1.1 Scopo del documento . . . 3

1.2 Glossario . . . 3

1.3 Riferimenti . . . 3

2 Processi primari **3**

2.1 Fornitura . . . 3

2.2 Sviluppo 4

3 Processi di supporto **4**

3.1 Documentazione 4

3.2 Gestione della configurazione 6

3.3 Verifica 9

3.4 Gestione della qualità e validazione 9

4 Processi organizzativi **10**

4.1 Gestione organizzativa 10

4.2 Formazione 11

1 Introduzione

1.1 Scopo del documento

Il documento raccoglie le regole di lavoro (*way of working*) che Ampalis Team adotta dalla propria costituzione: strumenti, convenzioni e procedure che tutti i componenti applicano allo stesso modo. Ogni regola nasce da una decisione presa in riunione e registrata in un verbale.

Il documento è organizzato per processi: processi primari, di supporto e organizzativi. Le parti non ancora necessarie sono indicate come da definire e saranno normate prima che le attività corrispondenti abbiano inizio.

1.2 Glossario

I termini specialistici saranno definiti nel *Glossario*, che il gruppo redigerà dopo l'aggiudicazione.

1.3 Riferimenti

Riferimenti normativi

- T. Vardanega, *Regolamento del Progetto Didattico*, a.a. 2026/2027, versione del 2026-10-11.

Riferimenti informativi

- ISO 8601, *Date and time – Representations for information interchange*.
- T. Preston-Werner, *Semantic Versioning 2.0.0*, `https://semver.org/lang/it/` (consultato il 2026-10-27).
- Verbali interni di Ampalis Team del 2026-10-19, del 2026-10-21, del 2026-10-23, del 2026-10-28, del 2026-10-29 e del 2026-11-11.
- T. Vardanega, *Aggiudicazione Appalti – I Lotto*, a.a. 2026/2027, 2026-11-03: rilievi sul versionamento e sull'approvazione per il rilascio.

2 Processi primari

2.1 Fornitura

Il processo regola i rapporti con il committente (i docenti) e con il proponente.

Comunicazioni ufficiali.

1. L'indirizzo ampalisteam@gmail.com è l'unico recapito ufficiale del gruppo verso docenti e proponenti: ogni comunicazione esterna parte da questo indirizzo.
2. Le consegne ai docenti avvengono per email, senza allegati, con il collegamento al repository e alla vista web, dove la *Lettera di Presentazione* è in evidenza.

Incontri con il proponente.

1. Il gruppo richiede l'incontro per email, inviando in anticipo ordine del giorno e domande.

2. Prima dell'incontro il gruppo stabilisce chi redige il verbale e raccoglie le domande in un documento di lavoro, divise per blocchi e con priorità. Durante l'incontro le domande si pongono a nome del gruppo: il verbale esterno non indica chi le pone.
3. Dopo l'incontro il redattore scrive il verbale esterno con il modello del gruppo; un altro componente lo verifica e il Responsabile ne approva il rilascio (sezione 3.1).
4. Approvato per il rilascio, nella versione 1.0.0, il verbale esce con il campo Conferma "In attesa di conferma" e viene inviato al proponente, che lo firma nell'ultima sezione.
5. La firma ricevuta entra in una nuova versione del verbale, verificata e registrata nel registro delle modifiche: il campo Conferma riporta l'azienda e la data della conferma, l'ultima sezione riporta nome, ruolo e data scritti dall'azienda e la firma, ritagliata dal PDF firmato e riportata con la stessa dimensione e posizione. Il PDF firmato dall'azienda si conserva nel repository `documenti_bozza`.

2.2 Sviluppo

Da definire dopo l'aggiudicazione: analisi dei requisiti (compreso lo stato dell'arte richiesto dal proponente), progettazione, codifica, integrazione e rilascio.

3 Processi di supporto

3.1 Documentazione

Strumenti e modelli. I documenti si scrivono in LaTeX, con un preambolo comune e uno stile grafico ricavato dal logo (file `template/preambolo.tex`). Ogni nuovo documento parte da un modello del gruppo, pubblicato in PDF nella cartella `template` del repository `documentazione`:

- `modello-documento.tex` per i documenti (per esempio *Valutazione dei Capitolati*, *Norme di Progetto*);
- `modello-verbale-interno.tex` e `modello-verbale-esterno.tex` per i verbali;
- `modello-diario-di-bordo.pptx` per le diapositive del diario di bordo.

Diapositive del diario di bordo. Il diario di bordo lo prepara il Responsabile, che lo espone in aula, e non si verifica (sezione 3.3). Le diapositive si preparano con il modello del gruppo in PowerPoint, o in Keynote, che apre lo stesso modello, e si caricano nel repository in formato PDF. LaTeX permetterebbe di produrre anche diapositive, ma per una presentazione breve, da preparare ogni settimana, un programma di presentazione è più rapido e permette a tutti i componenti di modificarla direttamente.

Struttura di un documento.

1. Copertina, con titolo, versione, data, stato, uso (interno o esterno), destinatari, redazione, verifica e approvazione, sempre con nome e cognome.
2. Registro delle modifiche, prima dell'indice.
3. Indice e, se il documento contiene tabelle, indice delle tabelle.
4. Corpo del documento: tabelle e figure sono numerate e hanno una didascalia.

Versionamento. Tutti i documenti, verbali compresi, hanno una versione secondo il versionamento semantico X.Y.Z e un registro delle modifiche. Fanno eccezione la *Lettera di Presentazione*, che ha solo la data perché accompagna una sola consegna e la versione inviata sostituisce le bozze, e il diario di bordo, che è un supporto all'esposizione in aula.

1. Ogni modifica di un documento, compresa la prima stesura, ha un solo autore e, di norma, un solo verificatore, persone diverse (sezione 3.3), e diventa una riga del registro solo dopo una verifica con esito positivo. La riga riporta versione, data, autore, verificatore e descrizione; le righe vanno dalla più recente alla più vecchia.
2. La prima stesura verificata ha versione 0.1.0. Ogni modifica verificata successiva alza Y (aggiunte o modifiche di contenuto) o Z (piccole correzioni) e azzera le cifre che seguono.
3. L'approvazione per il rilascio è un atto del solo Responsabile, su un documento già verificato, e non si verifica; si aggiunge nella stessa pull request della modifica verificata (sezione 3.2). Alza X, azzera Y e Z e si registra con una riga propria: il Responsabile come autore, nessun verificatore e la descrizione "Approvazione per il rilascio". L'approvazione non modifica il contenuto: se serve una correzione, la si fa e la si verifica prima.
4. Il primo rilascio ha quindi versione 1.0.0. Dopo il rilascio le modifiche verificate alzano Y o Z (per esempio 1.1.0 o 1.0.1); X sale solo con l'approvazione del rilascio successivo, che il Responsabile aggiunge nella pull request dell'ultima modifica del documento prima della consegna in cui è presentato, per esempio la Requirements and Technology Baseline (RTB). X sale quindi solo per un documento modificato dopo l'ultima approvazione, cioè con Y o Z diversi da zero: un documento ancora alla versione X.0.0 non cambia e si presenta così anche alle consegne successive. Alla consegna si presenta quanto è su `main`.
5. Due stati diversi di un documento non hanno mai lo stesso numero di versione: ogni modifica, anche minima, ha la sua versione e la sua riga.
6. La vista web legge la versione dalla copertina del PDF (sezione 3.2): la versione si scrive solo nel documento, in copertina e nella prima riga del registro, che devono coincidere.

Con la versione 1.2.0 di questo documento i registri dei documenti già pubblicati sono stati riordinati secondo queste regole, dopo l'esito dell'aggiudicazione: le modifiche verificate prima del rilascio sono passate alle versioni 0.x e la versione 1.0.0 registra solo l'approvazione per il rilascio.

Verbali. La prima stesura verificata di un verbale ha versione 0.1.0; con l'approvazione per il rilascio del Responsabile, nella stessa pull request della verifica (sezione 3.2), il verbale passa alla versione 1.0.0 ed è pubblicato. Ogni modifica successiva, verificata, aggiunge una riga al registro.

1. Ogni verbale riporta: informazioni sulla riunione, ordine del giorno, svolgimento, decisioni, azioni e prossima riunione.
2. Le decisioni hanno un codice (VI<n>-D<nn> nei verbali interni, VE<n>-D<nn> in quelli esterni) e riprendono quanto scritto nello svolgimento.
3. Ogni azione deriva da una decisione, ha assegnatario e scadenza ed è tracciata da una issue, il cui numero compare nel verbale.
4. Ogni verbale nuovo resta coerente con i precedenti: ciò che un verbale precedente ha già registrato si richiama, non si presenta come nuovo.

Lingua. Il repository e i documenti sono in italiano, lingua madre di tutti i componenti e lingua dei rapporti con il proponente: scrivere nella stessa lingua riduce le ambiguità all'interno

del gruppo e con l'azienda. Restano in inglese i termini standard degli strumenti, che non si traducono: le etichette predefinite di GitHub (per esempio `bug`, `documentation`, `enhancement`) e i campi e le colonne della Project Board (per esempio Backlog, In progress, Done).

Date nel testo e nei messaggi di commit. Le date si scrivono nel formato AAAA-MM-GG, per esempio 2026-10-21 (formato esteso della norma ISO 8601).

- È un formato internazionale e non ambiguo: una data come 03/04/2027 si legge come 3 aprile in Italia e come 4 marzo nei paesi anglosassoni, mentre 2027-04-03 ha una sola lettura.
- Va dall'unità più grande alla più piccola, quindi l'ordine alfabetico coincide con quello cronologico, per esempio negli elenchi di commit e nelle tabelle.
- I trattini separano anno, mese e giorno e rendono la data leggibile a colpo d'occhio.

Nomi dei file. I nomi dei file sono in minuscolo, con le parole separate da trattini. Se il nome contiene una data, la data viene per prima, nel formato AAAAMMGG (formato base della norma ISO 8601, senza separatori), ed è separata dal resto del nome con un trattino basso: per esempio `20261021_verbale-interno.pdf`. Hanno la data nel nome solo i verbali (`verbale-interno` e `verbale-esterno`), i diari di bordo (`diario-di-bordo`) e la *Lettera di Presentazione* (`lettera-di-presentazione`), sempre con questi nomi: la vista web ne ricava il tipo (sezione 3.2).

- L'ordine alfabetico dei file nelle cartelle e nel repository coincide con quello cronologico, come nel testo.
- Senza trattini la data è un unico blocco di cifre, distinto dal trattino che nei nomi separa le parole (`verbale-interno`) e dal trattino basso che separa la data dal resto del nome. Il nome resta più corto e la data si seleziona e si cerca come un'unica parola.
- Nel testo, al contrario, si usano i trattini, perché lì conta la leggibilità.

Regole di scrittura.

- I nomi dei documenti si scrivono in corsivo e con le iniziali maiuscole (per esempio *Dichiarazione degli Impegni*); in corsivo vanno anche i nomi di issue e milestone (per esempio *Candidatura*).
- Nei titoli di sezione va in maiuscolo solo la prima parola, salvo nomi propri e nomi di documenti; gli acronimi si sciolgono alla prima occorrenza.
- Il testo da completare o confermare si scrive in rosso; nessun testo rosso deve restare in un documento rilasciato.

3.2 Gestione della configurazione

Strumenti. Il gruppo usa git per il versionamento e GitHub per ospitare i repository, raccolti nella GitHub organization del gruppo (`https://github.com/Ampalis-Team/`). Tutto il materiale del progetto, bozze comprese, resta su GitHub: il gruppo non scambia documenti su altri canali, come Google Drive, WhatsApp o Discord.

Repository	Visibilità	Contenuto
`documentazione`	Pubblico	Documenti del progetto pronti per la verifica e rilasciati, consultabili da committente e proponente
`documenti_bozza`	Privato	Bozze dei documenti, condivise tra i componenti mentre sono in lavorazione, e materiale di lavoro, come i PDF dei verbali firmati dalle aziende
`progetto_ingegneria_del_software_2026-27`	Pubblico	Codice del Proof of Concept e del Minimum Viable Product; per ora è vuoto e ha un nome generico

Una bozza resta nel repository privato finché il suo autore la considera pronta; a quel punto entra nel repository `documentazione` con una pull request, secondo i passi del paragrafo sulle pull request.

Per scelta del gruppo, nel repository `documentazione` i documenti si pubblicano già in PDF e non in sorgente LaTeX, così chi consulta il repository legge direttamente il documento finale. Lo stesso vale per i modelli della cartella `template`, pubblicati in PDF perché chiunque possa consultarli senza compilarli.

Struttura del repository della documentazione. Il repository `documentazione` è organizzato in cartelle secondo il tipo e l'uso dei documenti, non secondo la consegna a cui appartengono: così un documento che evolve nel tempo resta sempre nella stessa cartella.

```
documentazione/
|-- documentazione-esterna/   # documenti per committente e proponente
|-- documentazione-interna/   # documenti a uso del gruppo
|-- template/                 # modelli dei documenti
|-- logo/                     # logo del gruppo
|-- vista-web/                # generatore e consegne della vista web
`-- .github/workflows/        # pubblicazione della vista web
```

- `documentazione-esterna` raccoglie i documenti rivolti all'esterno: i documenti delle candidature e delle revisioni (per esempio *Valutazione dei Capitolati*, *Dichiarazione degli Impegni*, *Lettera di Presentazione*), i verbali esterni e i diari di bordo.
- `documentazione-interna` raccoglie i documenti a uso del gruppo: i verbali interni e le *Norme di Progetto*.
- `template` contiene i modelli dei documenti, dei verbali e del diario di bordo, in PDF.
- `logo` contiene il logo del gruppo, a colori e in bianco e nero.
- `vista-web` contiene il generatore della vista web e il file `documenti.toml`, con i testi della pagina e le consegne; `.github/workflows` contiene la pubblicazione automatica.
- Nella radice, un file di presentazione descrive il repository e riporta l'indirizzo della vista web.

I nomi dei file seguono le regole della sezione 3.1.

Vista web. I documenti pubblicati si consultano anche dalla vista web del gruppo, `https://ampalis-team.github.io/documentazione/`, una pagina su GitHub Pages che li raccoglie per consegna, con la *Lettera di Presentazione* in evidenza, divisi in documentazione esterna e interna e con verbali e diari di bordo dal più recente.

- La pagina si rigenera a ogni modifica di `main`, a partire dai PDF presenti nelle cartelle `documentazione-esterna` e `documentazione-interna`: un documento compare quando il suo PDF è su `main`, cioè dopo l'unione della pull request. La generazione non modifica il repository e non fa commit.
- Ogni informazione si ricava dal PDF e dal suo nome: tipo e uso dalla cartella e dal nome del file; la data di verbali, diari di bordo e *Lettera di Presentazione* dal nome del file; la versione dalla copertina; il titolo dei documenti dai metadati del PDF, che il preambolo comune imposta, e l'azienda dei verbali esterni dai destinatari in copertina.
- Verbali, diari di bordo e *Lettera di Presentazione* compaiono nella prima consegna con data uguale o successiva alla loro. Un documento compare nella consegna della sua ultima approvazione per il rilascio, letta nel registro delle modifiche, e passa da solo alla consegna successiva quando viene approvato di nuovo; un documento mai approvato compare nella prima consegna successiva alla data in copertina.
- Il file `documenti.toml` della cartella `vista-web` non elenca i documenti: contiene solo i testi della pagina e le consegne, con nome e data, e si modifica solo quando questi cambiano.
- Su ogni pull request un controllo automatico segnala: i nomi di file che non seguono le regole della sezione 3.1; i documenti e i verbali senza versione in copertina, con copertina e prima riga del registro diverse o con una versione X.Y.Z, con X almeno 1, senza la riga di approvazione per il rilascio della X.0.0; più di una *Lettera di Presentazione* nella stessa consegna; i PDF già su `main` modificati senza una versione più alta (punto 5 del versionamento). La pull request si unisce solo con il controllo superato.

Rami. Il ramo `main` è protetto: si modifica solo con una pull request approvata da un componente diverso dall'autore e con il controllo della vista web superato. L'approvazione resta valida anche dopo il commit di approvazione per il rilascio del Responsabile, descritto nel paragrafo sulle pull request. L'unica eccezione è la pull request del diario di bordo, che il Responsabile unisce senza approvazione, perché il diario non si verifica (sezione 3.3): solo il Responsabile può aggirare la protezione, solo in questo caso e sempre con il controllo della vista web superato. Push forzati e cancellazione del ramo sono bloccati. Il lavoro si svolge in rami dedicati, che restano nel repository anche dopo l'unione, come storico.

Issue. Ogni attività è tracciata da una issue.

1. La issue ha uno o più assegnatari, le etichette pertinenti e la milestone a cui contribuisce.
2. Se nasce da un verbale, la descrizione riporta l'origine (verbale, azione e decisione) e la scadenza.
3. Ogni issue compare nella Project Board, dove se ne segue l'avanzamento (sezione 4.1).
4. Per quanto possibile, la issue si collega a ciò che la completa con una parola chiave di chiusura (per esempio `Closes #8`): alla pull request, se richiede più modifiche, oppure al singolo commit, se basta una modifica.

Pull request. Ogni prodotto entra in `main` con una sola pull request, che contiene stesura, verifica e, quando è prevista, approvazione per il rilascio, in questi passi.

1. Il redattore fa il commit del prodotto, con la nuova versione e la sua riga nel registro (per esempio la 0.1.0 di un verbale), e apre la pull request verso `main`, con un titolo che descrive il risultato e, nella descrizione, la parola chiave di chiusura della issue (per esempio `Closes #8`). Come revisore indica il verificatore e come assegnatario il Responsabile.

2. Il verificatore esamina il prodotto secondo la sezione 3.3 e, se l'esito è positivo, approva la pull request.
3. Se è prevista l'approvazione per il rilascio (sezione 3.1), come per un verbale o per l'ultima modifica di un documento prima di una consegna, il Responsabile la aggiunge nella stessa pull request, con un commit che contiene solo la nuova versione X.0.0 e la riga di approvazione. Dopo l'approvazione del verificatore non entrano altri commit: se serve una modifica, la fa l'autore e il verificatore approva di nuovo.
4. Il Responsabile unisce la pull request e la issue collegata si chiude in automatico. L'unione di una modifica verificata non è di per sé un'approvazione per il rilascio: lo è solo il commit X.0.0 del Responsabile.
5. La pull request del diario di bordo la apre e la unisce il Responsabile, senza revisori, perché il diario non si verifica: è l'unica eccezione alla protezione di `main` (paragrafo sui rami).

Commit.

- Ogni componente fa i commit con il proprio account GitHub e con nome e indirizzo email associati a quell'account.
- Ogni commit contiene una sola aggiunta o modifica, piccola e circoscritta.
- Il messaggio è in italiano e descrive il risultato: un participio passato, poi l'oggetto, l'eventuale data nel formato AAAA-MM-GG e il riferimento alla issue. Esempio: "Redatto verbale interno del 2026-10-21 - Closes #8".

3.3 Verifica

1. Ogni prodotto è verificato da un solo componente, diverso dal suo autore. Un secondo verificatore si aggiunge solo nei casi importanti, che il Responsabile indica nella issue; per il codice le regole dello sviluppo (sezione 2.2) prevederanno due verificatori per ogni funzionalità.
2. Il diario di bordo non si verifica: lo prepara il Responsabile, che lo espone, e riassume quanto già registrato nei verbali interni ed esterni, che sono verificati. Non si verifica nemmeno l'approvazione per il rilascio (sezione 3.1).
3. La verifica avviene nella revisione della pull request. Il verificatore controlla la correttezza del contenuto, il rispetto di questo documento e dei modelli, l'assenza di testo rosso e la coerenza con gli altri documenti, in particolare con i verbali precedenti. Controlla anche che la parola chiave di chiusura, nel commit e nella pull request, indichi la issue giusta.
4. La verifica è in sola lettura: il verificatore non modifica il prodotto che verifica. Se trova difetti, li segnala nella pull request e chiede le modifiche; l'autore le apporta, anche quando sono minime, e la verifica riprende dal punto 3.
5. Con esito positivo il verificatore approva la pull request; solo allora la modifica entra nel registro delle modifiche.

3.4 Gestione della qualità e validazione

Da definire dopo l'aggiudicazione, con il cruscotto di valutazione della qualità previsto dal regolamento e con il test book concordato con il proponente come criterio di accettazione del prodotto.

4 Processi organizzativi

4.1 Gestione organizzativa

Ruoli. Il gruppo adotta i ruoli del regolamento: Responsabile, Amministratore, Analista, Progettista, Programmatore e Verificatore. Nel periodo di candidatura il Responsabile è stato Ivan Valente, mentre redattori e verificatori dei documenti ruotano tra gli altri componenti. Dopo l'aggiudicazione i ruoli ruotano secondo le regole della *Dichiarazione degli Impegni*: un solo ruolo per persona alla volta, tutti i ruoli per ogni componente e nessuna verifica dei propri prodotti.

Comunicazione interna. Il gruppo usa Discord per le riunioni e un gruppo WhatsApp per le comunicazioni rapide.

Calendario condiviso. Riunioni, scadenze e indisponibilità sono raccolte in un calendario di Google Calendar creato appositamente per il gruppo.

1. A ogni evento sono invitati tutti i componenti, al loro indirizzo email istituzionale, per tenere separati gli impegni universitari da quelli privati.
2. Google Calendar avvisa tutti per email non appena un evento viene creato, modificato o cancellato, e con una notifica dell'app un giorno prima e un'ora prima dell'evento.
3. Ogni evento si ricorda anche nel gruppo WhatsApp, menzionando tutti i componenti.

Decisioni. Ogni decisione, anche la più piccola, si mette in pratica solo se tutti i componenti sono d'accordo, così da raccogliere sempre eventuali suggerimenti prima di agire.

1. Chi propone un'azione a nome del gruppo (per esempio l'invio di un'email) la scrive nel gruppo WhatsApp e menziona tutti i componenti con la funzione di menzione.
2. L'azione si svolge solo quando tutti hanno risposto in modo favorevole; finché manca anche una sola risposta, non si procede.
3. La stessa regola vale per le decisioni prese in riunione.

Riunioni interne.

1. Le riunioni interne si svolgono principalmente in chiamata online su Discord; almeno una riunione a settimana è fissa.
2. Le chiamate si registrano, così chi non riesce a partecipare può recuperarle e restare al passo.
3. Ogni riunione ha un ordine del giorno, che è il primo punto del verbale.
4. Un redattore, a rotazione, scrive il verbale interno con il modello del gruppo; un altro componente lo verifica e il Responsabile ne approva il rilascio.
5. Le azioni decise in riunione diventano issue, con assegnatario e scadenza.

Metodo di lavoro. Il gruppo segue il metodo agile Scrum.

1. Le attività si raccolgono in una Project Board della GitHub organization, condivisa tra il repository della documentazione e quello del progetto: riguarda il gruppo nel suo insieme e raccoglie tutto il lavoro, dalla candidatura alla fine del progetto.
2. La Project Board usa un modello ispirato a Scrum, con vista Kanban e colonne Backlog, Ready, In progress, In review e Done.

3. Le attività si organizzano in milestone (per esempio *Candidatura*) e, dopo l'aggiudicazione, in sprint, con la durata definita nella *Dichiarazione degli Impegni*.

Definition of Done. Da definire. Finora il gruppo ha prodotto solo documenti, per i quali un'attività è conclusa quando la modifica è verificata (salvo il diario di bordo) e unita al ramo `main` e, se il documento va rilasciato, quando il Responsabile ne ha approvato il rilascio. Per il codice la *Definition of Done* sarà stabilita dopo l'aggiudicazione, in base alle indicazioni del proponente e alla realtà di interesse.

4.2 Formazione

Da definire dopo l'aggiudicazione: piano di formazione sulle tecnologie del capitolato, con un referente per ciascuna tecnologia.